# Sedaro Recruiting Report

Charlotte Beckford

April 6, 2026

## 1 Simulation Sensitivity to Initial Conditions

I was interested in exploring the sensitivity of the simulation to initial conditions. There are many ways that two bodies moving through three-dimensional space are related to one another. At different times, we may be interested in the velocities of each separate object or their relative velocity with one another. For this initial exploratory analysis, I decided to focus on how changing the initial conditions affects the minimum and maximum distances between the two objects throughout the simulation. I was imagining that if the two objects are rotating bodies in space, it is incredibly important that they do not make contact with one another, hence why the minimum distance between them is an important output. Similarly, if one body is a satellite and the other is a satellite or a planetary object, it may be necessary for the two objects to be sufficiently close so that they can send transmissions back and forth or so that the satellite can take measurements of the planetary object, respectively. This consideration is why the maximum distance between the two bodies is an output of interest.

The simulation function accepts the following initial data:

```
data = {
    'Body1': {
        'timeStep': tS1,
        'time': t1,
        'position': {'x': pos1x, 'y': pos1y, 'z': pos1z},
        'velocity': {'x': vel1x, 'y': vel1y, 'z': vel1z},
        'mass': m1
    },
    'Body2': {
        'timeStep': tS2,
        'time': t2,
        'position': {'x': pos2x, 'y': pos2y, 'z': pos2z},
        'velocity': {'x': vel2x, 'y': vel2y , 'z': vel2z},
        'mass': 0.123
    }.
```

Hence, our initial conditions are:

- $\tau_1, \tau_2$, the `timeStep` of `Body1` and `Body2`, respectively
- $T_{0,1}, T_{0,2}$ the `time` of `Body1` and `Body2`, respectively
- $\hat{r}_1 = \begin{bmatrix} \hat{r}_{1,x} \\ \hat{r}_{1,y} \\ \hat{r}_{1,z} \end{bmatrix}$, the `position` of `Body1`
- $\hat{r}_2 = \begin{bmatrix} \hat{r}_{2,x} \\ \hat{r}_{2,y} \\ \hat{r}_{2,z} \end{bmatrix}$, the `position` of `Body2`
- $\hat{v}_1 = \begin{bmatrix} \hat{v}_{1,x} \\ \hat{v}_{1,y} \\ \hat{v}_{1,z} \end{bmatrix}$, the `velocity` of `Body1`
- $\hat{v}_2 = \begin{bmatrix} \hat{v}_{2,x} \\ \hat{v}_{2,y} \\ \hat{v}_{2,z} \end{bmatrix}$, the `velocity` of `Body2`
- $m_1, m_2$ the `mass` of `Body1` and `Body2`, respectively.

To me, it made sense to always have $T_{0,1}, T_{0,2} = 0$ and for $\tau_1 = \tau_2$. In real data, I can imagine that we are given the positions and velocities of two bodies at the same initial time and that measurements are taken periodically at the same time points.

Varying $\hat{r}_1$ and $\hat{r}_2$ simultaneously seemed superfluous to me since the frame of reference of the initial conditions could always be reset so that `Body1` is at the position $\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$.

I conducted a Sobol global sensitivity analysis to investigate which of the initial conditions $\hat{r}_2$, $\hat{v}_1$, $\hat{v}_2$, $m_1$, and $m_2$ accounts for the largest variation in the outputs of interest, the maximum distance between the two objects during the simulation, and the minimum distance of the two objects over the simulation.

First, I wrote the Python script `ICSimulation.py` which accepts as input the relevant initial conditions and returns my outputs of interest, the minimum and maximum distances between the two objects over the simulation. To compute the distances between the two objects at a time point, I computed the L2 norm using the `np.linalg.norm()` function of the difference between the two positions at that time point, $\hat{r}_1 - \hat{r}_2$.

Then I cribbed a Python script I use in my doctoral research to compute Sobol indices to compute Sobol indices of the initial conditions under consideration for my outputs of interest. The script `SobolCheckpoints.py` is written so that the data is regularly saved in case of keyboard interruption. To call the script, you pass in the number of samples to use to compute the Sobol indices, `N`. You may also optionally pass in the number of cores to use as the script is written for parallel implementation, but the script will default to using the

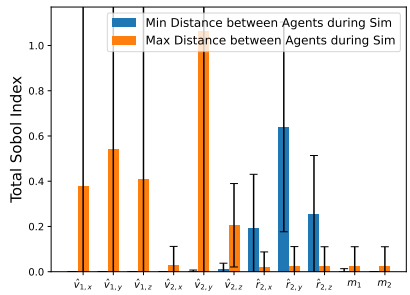


Figure 1: Total-order Sobol indices of the initial conditions with number of samples $N = 4$.

number of cores available on the cpu. Finally, there are two modes with which the script can be called. Passing the argument `-m generate` generates the data to compute the Sobol indices. This takes up the bulk of the computation time. Passing the argument `-m analyze` computes the Sobol indices and produces a pdf image displaying total-order indices. A larger total-order index for a parameter means that varying that parameter has greater influence on the output of consideration, so in the image, we attribute the quantities with the largest bars to have the greatest influence on the outputs of interest.

Bounds of a uniform distribution from which the initial conditions are sampled must be specified in the code. From looking at the code provided to me, velocity and position can have negative or positive values. It makes sense to me that mass can only be positive. I decided to simply sample the masses from $U([0, 100])$ and all other quantities from $U([-100, 100])$.

The hyperparameter `N` should always be a power of 2. Increasing `N` decreases the confidence intervals of the total-order Sobol indices, generating more precise measurements. As increasing `N` also increases the run time of the `generate` mode, we want to optimize `N` so that it is large enough to generate sufficiently precise measurements but small enough that the runtime is feasible. We want to see convergence of the total-order Sobol indices. That is, if we increase `N` by a factor of 2 and the indices computed are roughly the same, we have accurate enough computations of our Sobol indices.

Initially, I ran the code with `-N 4` simply to get an idea of the run-time. The `-m generate` run ran in 11 seconds and `-m analyze` produced Figure 1.

The confidence intervals are extremely large- some even have negative lower bounds, which is very infeasible as total order Sobol indices should always be non-negative. This was simply to get an estimate of the runtime, and these estimates of the total-order Sobol indices should not be used to interpret which initial conditions have the greatest influence on these outputs.

I then ran `python SobolCheckpoints.py -N 512 -n 4 -m generate`. I expected this to run for 24 minutes. This was a fairly accurate estimate; the run-

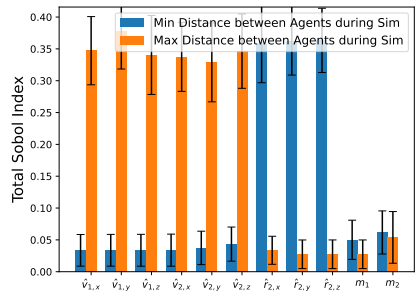


Figure 2: Total-order Sobol indices of the initial conditions with number of samples $N = 512$.

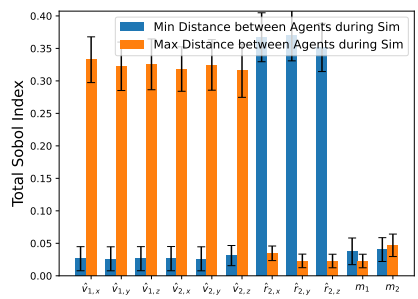


Figure 3: Total-order Sobol indices of the initial conditions with number of samples $N = 1024$.

time was 22 minutes. The command `python SobolCheckpoints.py -N 512 -n 4 -m analyze` generated Figure 2.

The confidence intervals are quite small, so I trust that these are decent approximation of the total Sobol indices. For thoroughness, I re-ran the computation of the total Sobol indices with double the number of samples (`-N 1024`) to verify convergence of the estimates of the Sobol indices. I expected this to run for 47 minutes. This was a fairly accurate estimate; the runtime was 46 minutes. The command `python SobolCheckpoints.py -N 1024 -n 4 -m analyze` generated Figure 3.

The confidence intervals produced by $N = 1024$ are smaller than those produced by $N = 512$, as expected. However, there is a good match between the point-wise estimates of the total order Sobol indices of both computations. This indicates that I have used enough samples to generate accurate total order Sobol indices.

From this Sobol analysis, I see that the velocities of both bodies are very

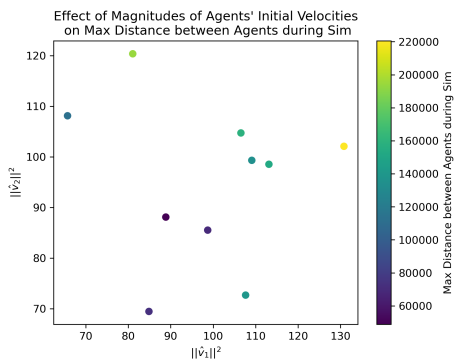


Figure 4: Grid search visualizing the maximum distance of agents during simulation to the magnitudes of their initial velocities with 10 samples.

influential on the maximum distance between the two agents during the simulation, but not on the minimum distance between the agents during the simulation. The position of the second object is influential on the minimum distance between the two agents during the simulation, but not on the maximum instance between the agents during the simulation. Neither mass of either object was very influential on either of these outputs. While this may seem strange, I was wondering if this could be attributed to the fact that in this code mass is not propagated.

The total order Sobol indices inform me which initial conditions are most influential on the minimum and maximum distances between the two agents, but not the relationship between these initial conditions and these outputs. I ran a grid-search to gain an idea of these relationships. Initially I ran this grid search for $n = 10$ to gain an idea of the runtime. Calling

```
python GridSearchICSimulation.py -n 10
```

took 3 seconds to run and produced Figures 4

Figure 4 is a visualization of how the maximum distance between the agents during the simulation changes for varying the magnitude ($L^2$-norm) of the initial velocities of each object.

Figure 4 is a visualization of how the minimum distance between the agents during the simulation changes for varying the initial position of `Body2`.

I then ran the same code with `-n 1000`. I expected it to take about 5 minutes to run. It took about 5.5 minutes and produced Figures 6 and 7.

I see that in the 2-dimensional graphic the color becomes lighter along the $y = x$ axis. Furthermore, the color is roughly symmetric across this axis. This means that, regardless of which bodies velocity we are considering, increasing the magnitude of the velocity results in a larger maximum distance between the agents during the simulation.

In the 3-dimensional graphic I see that the color is darker towards the origin $(0, 0, 0)$ and lighter as we move radially away from the origin. Recalling that

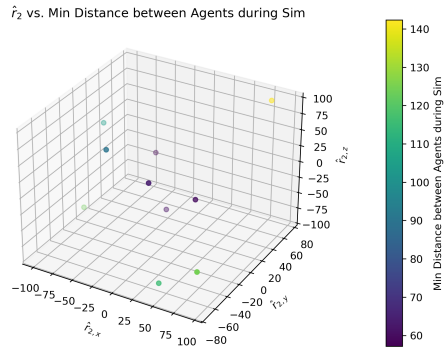


Figure 5: Grid search visualizing the minimum distance of agents during simulation to the initial position of the second agent with 10 samples.

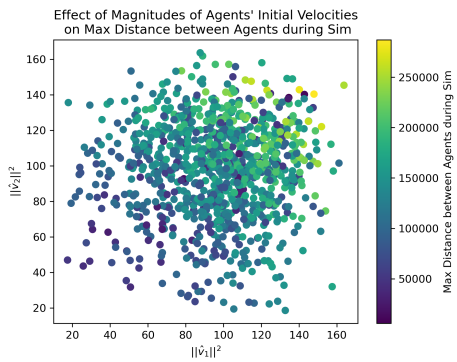


Figure 6: Grid search visualizing the maximum distance of agents during simulation to the magnitudes of their initial velocities with 1000 samples.

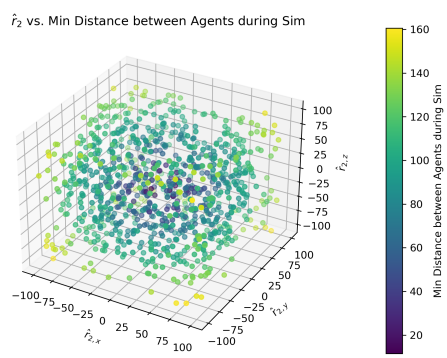


Figure 7: Grid search visualizing the minimum distance of agents during simulation to the initial position of the second agent with 1000 samples.

the position of `Body1` was fixed at the origin, I infer that increasing the initial distance between the two bodies results in a larger minimum distance between the agents during simulation. I suppose that since the movement of the two objects is controlled by the law of gravitation, they are attracted to each other over the course of the simulation, so the minimum value may always be occurring at the beginning of the simulation. Including repulsion between the two objects would likely change these results.

## 2 Mass Propagation

Since the mass of the two agents was constant throughout time, I wanted to include a mass propagation grounded in reality. Initially, I thought that I could have the mass update at discrete time points to represent loading or unloading of passengers or cargo, or I could have the mass deplete linearly to represent fuel burning. Upon a brief search, I decided to attempt to represent mass propagation by the Rocket Law[1], which states that given the initial mass of an object $\boldsymbol{v}_i$, the current velocity of an object $\boldsymbol{v}$,the velocity at which the engines of a rocket eject gas $\boldsymbol{u}$, and the initial mass $m_i$, then the current mass of the object $m$ is

$$m = m_i \exp\left(\frac{\boldsymbol{v}_i - \boldsymbol{v}}{\boldsymbol{u}}\right). \tag{1}$$

Since I am in the three-dimensional case, I made $\boldsymbol{v}_i - \boldsymbol{v} = ||\boldsymbol{v}_i - \boldsymbol{v}||^2$. Due to time constraints, I obviated having $\boldsymbol{u} = -1\boldsymbol{i}$ (the reference I looked at had in general $\boldsymbol{u} = -u\boldsymbol{i}$). My code to propagate the mass is as follows

```
    def propagate_mass(oldMass, oldVel, vel):
mass = oldMass*np.exp(-1*np.linalg.norm(np.array(list(oldVel.values()))-
                                np.array(list(vel.values()))))
return mass
```

and I updated the cells regarding mass propagation in the agents to be:

```
        {
         ‘consumed’: ‘‘‘(
             prev!(mass),
             prev!(velocity),
             velocity,
         )’’’,
         ‘produced’: ‘‘‘mass.’’’,
         ‘function’: propagate_mass,
    }.
```

The script which contains the updated mass propagation is `modsimPropMass`. I made a companion `simulatorPropMass` to call it. I compared the masses of

the two objects without mass propagation and with the mass propagation in `testPropMass`. The output was:

```
Without Mass Propagation:

Mass of Body1:
{100.0: 1, 200.0: 1, 300.0: 1, 400.0: 1, 500.0: 1, 600.0: 1,
700.0: 1, 800.0: 1, 900.0: 1, 1000.0: 1, 1100.0: 1}

Mass of Body2:
{100.0: 0.123, 200.0: 0.123, 300.0: 0.123, 400.0: 0.123, 500.0: 0.123, 600.0: 0.123,
700.0: 0.123, 800.0: 0.123, 900.0: 0.123, 1000.0: 0.123, 1100.0: 0.123}


Propagating Mass with Rocket Law:

Mass of Body1:
{100.0: np.float64(0.9999993397537946), 200.0: np.float64(0.9967035617171538),
300.0: np.float64(0.9933377867834169), 400.0: np.float64(0.9898946948639514),
500.0: np.float64(0.9863799786395882), 600.0: np.float64(0.9828162536274283),
700.0: np.float64(0.9792451021211063), 800.0: np.float64(0.9757246733487229),
900.0: np.float64(0.9723213221600688), 1000.0: np.float64(0.9690970097756206),
1100.0: np.float64(0.9660974776868391)}

Mass of Body2:
{100.0: np.float64(0.12299966987723138), 200.0: np.float64(0.11974235628428431),
300.0: np.float64(0.11641790685971441), 400.0: np.float64(0.1130194253144367),
500.0: np.float64(0.10955273506567134), 600.0: np.float64(0.10604010362324848),
700.0: np.float64(0.1025221279734845), 800.0: np.float64(0.0990553269746991),
900.0: np.float64(0.09570402999804492), 1000.0: np.float64(0.09252824676913116),
1100.0: np.float64(0.08957230901332702)}
```

The masses of both objects are now decreasing over time due to fuel being expelled, as expected.

# 3 Future Work

My analysis and implementation of mass propagation were extremely preliminary. There are many future directions I would have pursued with more time.

I would have liked to visualize the maximum distance with respect to eh actual coordinates of the velocities, rather than just their magnitudes. I would have liked to conduct regression analyses to verify the trends I observed from my visualizations- namely the linear relationship between the maximum of the magnitudes of the initial velocities of the two objects and the maximum distance between them and the positive linear relationship between the distance

of the initial positions of the bodies and their minimum distances during the simulation.

I could further explore the parameter space of these initial conditions using Latin Hypercube Sampling. This is an efficient sampling algorithm for high-dimensional parameter spaces because it avoids generating samples clustered together.

Given more time, I would have adjusted the code so that $\boldsymbol{u}$ is an input for each agent, that is, the velocity at which each body ejects fuel. I also would have liked to repeat the exploration I conducted exploring how varying initial conditions varied the simulations exploring the effects of varying the initial masses and these initial $\boldsymbol{u}$s when the mass is being propagated.

# References

[1] Lumen Learning. Rocket propulsion, n.d. URL `https://courses.lumenlearning.com/suny-osuniversityphysics/chapter/9-7-rocket-propulsion/` Accessed: 2026-04-06.